1. Documentación Técnica – API de Gestión RFID

1.1 Información General

- **URL Base:** `http://tu-servidor:3000/api`
- **Formato de Respuesta:** JSON
- **Codificación:** UTF-8
- **Autenticación:** API Key enviada en el header `x-api-key`

Ejemplo de Header:

```
x-api-key: tu_api_key_aqui
```

- **Versión de la API:** 1.0.0
- **Última actualización:** Enero 2024

1.2 Autenticación

- Todas las llamadas requieren una API Key válida.
- **Obtener API Key (solo desarrollo local):**

```
GET /api/config
```

> Nota: Solo accesible desde redes locales por seguridad.

1.3 Endpoints Disponibles

1.3.1 EPC_READINGS

Consultar Stock Actual

```
GET /api/epc-readings/stock
```

- Parámetros opcionales: `devid`, `startDate`, `endDate`
- **Respuesta:**

```
[
  { "devid": "ALM001", "EPC": "E2000016600001234567890 1", "Local_time": "2024-01-
15T10:30:00.000Z" }
]
```

Consultar Stock Temporal

```
GET /api/epc-readings/stock-temporal
```

- Parámetros opcionales: `devid`, `startDate`, `endDate`

- **Respuesta:**

```
[
  { "devid": "ALM001", "EPC": "E2000016600001234567890 1", "Local_time": "...", "EPC_User":
"...", "Time_User": "..." }
]
```

Eliminar Stock Temporal

```
DELETE /api/epc-readings/stock-temporal
```

- **Respuesta:**

```
{ "message": "X registros de stock temporal eliminados correctamente" }
```

1.3.2 BIN (Consumo)

Consultar Stock Consumido

```
GET /api/bin/stock-consumption
```

- Parámetros: `devid`, `patient`, `operating_room`, `startDate`, `endDate`
- **Ejemplo:** `/api/bin/stock-consumption?patient=PAC001&operating_room=QUI01`

Eliminar Todos los Registros

```
DELETE /api/bin
```

- **Respuesta:** `{ "message": "X registros de Bin eliminados correctamente" }`

1.3.3 USER_READINGS

Consultar Usuarios en Puertas

```
GET /api/user-readings/user-door
```

- Parámetros opcionales: `devid`, `startDate`, `endDate`

Eliminar Todos los Registros

```
DELETE /api/user-readings
```

1.3.4 USER_ROOM

- **Listar todos los registros:** `GET /api/user-room`
- **Buscar por EPC:** `GET /api/user-room/epc/{epc}`
- **Buscar por Puerta:** `GET /api/user-room/door/{door}`
- **Crear Registro:** `POST /api/user-room`
- **Actualizar Registro:** `PUT /api/user-room/{id}`

- **Eliminar Registro:** `DELETE /api/user-room/{id}`
- **Eliminar todos los registros:** `DELETE /api/user-room`

1.3.5 EPC_MASTER

- **Listar todos:** `GET /api/epc-master`
- **Buscar por EPC:** `GET /api/epc-master/epc/{epc}`
- **Buscar por descripción:** `GET /api/epc-master/description/{description}`
- **Buscar por almacén:** `GET /api/epc-master/almacen/{almacen}`
- **Crear registro:** `POST /api/epc-master`
- **Actualizar registro:** `PUT /api/epc-master/{id}`
- **Eliminar registro:** `DELETE /api/epc-master/{id}`

1.4 Códigos de Respuesta HTTP

Código	Significado
200 OK	Operación exitosa
201 Created	Recurso creado
400 Bad Request	Parámetros inválidos
401 Unauthorized	API Key no proporcionada
403 Forbidden	API Key inválida
404 Not Found	Recurso no encontrado
500 Internal Server Error	Error del servidor

1.5 Formato de Errores

```
{
  "error": {
    "message": "Descripción del error",
    "status": 400,
    "timestamp": "2024-01-15T10:30:00.000Z"
  }
}
```

1.6 Ejemplos de Implementación

JavaScript (Fetch API)

```
async function apiRequest(endpoint, method='GET', data=null) {
  const options = { method, headers: { 'x-api-key': API_KEY, 'Content-Type':
'application/json' } };
```

```
  if (data && ['POST','PUT'].includes(method)) options.body = JSON.stringify(data);
  const res = await fetch(`${API_BASE_URL}${endpoint}`, options);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

Python (requests)

```
def api_request(endpoint, method='GET', data=None):
    url = f"{API_BASE_URL}{endpoint}"
    if method=='GET': r=requests.get(url, headers=headers)
    elif method=='POST': r=requests.post(url, headers=headers, json=data)
    elif method=='PUT': r=requests.put(url, headers=headers, json=data)
    elif method=='DELETE': r=requests.delete(url, headers=headers)
    r.raise_for_status()
    return r.json()
```

PHP (cURL)

```
function makeRequest($endpoint,$method='GET',$data=null){
  $curl=curl_init();
  curl_setopt_array($curl,[
    CURLOPT_URL=>$this->baseUrl.$endpoint,
    CURLOPT_RETURNTRANSFER=>true,
    CURLOPT_HTTPHEADER=>['x-api-key: '.$this->apiKey,'Content-Type: application/json'],
    CURLOPT_CUSTOMREQUEST=>$method
  ]);
  if($data && in_array($method,['POST','PUT']))
curl_setopt($curl,CURLOPT_POSTFIELDS,json_encode($data));
  $response=curl_exec($curl); curl_close($curl);
  return json_decode($response,true);
}
```

1.7 Consideraciones de Rendimiento

1. Filtrar siempre por fechas y dispositivos para evitar grandes volúmenes de datos.
2. Implementar caché para datos inmutables.
3. Reutilizar conexiones HTTP y limitar llamadas concurrentes.
4. Considerar paginación para reportes extensos.

1.8 Consideraciones de Seguridad

1. Mantener la API Key segura; no exponerla en cliente web.
2. Usar HTTPS en producción.
3. Implementar rate limiting en aplicaciones cliente.
4. Validar todos los datos antes de enviarlos a la API.

---

2. Guía de Inicio Rápido – API RFID

2.1 Introducción

Permite empezar a usar la API de Gestión RFID en minutos. Cubriendo conexión, operaciones básicas, manejo de errores y ejemplos prácticos.

2.2 Paso 1: Obtener Acceso

- **URL Base:** `http://tu-servidor:3000/api`
- **Autenticación:** API Key (proporcionada por administrador)
- **Formato:** JSON

Prueba de Conectividad:

```
curl -X GET "http://tu-servidor:3000/api/epc-master" \
  -H "x-api-key: TU_API_KEY" \
  -H "Content-Type: application/json"
```

Respuesta Esperada:

```
[
  { "id": 1, "epc": "E2000016600001234567890 1", "short_description": "Bisturí quirúrgico",
"Local_time": "2024-01-15T10:30:00.000Z", "Almacen": "ALM001" }
]
```

2.3 Paso 2: Operaciones Básicas

Consultar Stock Actual

```
curl -X GET "http://tu-servidor:3000/api/epc-readings/stock" -H "x-api-key: TU_API_KEY"
```

Buscar Producto por EPC

```
curl -X GET "http://tu-servidor:3000/api/epc-master/epc/E2000016600001234567890 1" -H "x-api-
key: TU_API_KEY"
```

Crear Nuevo Producto

```
curl -X POST "http://tu-servidor:3000/api/epc-master" \
  -H "x-api-key: TU_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"epc":"E200001660
```

3. Casos de Uso Completos – API RFID

3.1 Gestión de Inventario

Escenario

Hospital o clínica necesita gestionar el inventario de instrumental quirúrgico con tags RFID, controlando disponibilidad y ubicación en tiempo real.

Flujo de Trabajo

1. **Registrar Nuevo Instrumental**

```
curl -X POST "http://servidor:3000/api/epc-master" \
  -H "x-api-key: API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "epc": "E2000016600001234567890 1",
    "short_description": "Bisturí quirúrgico #15",
    "almacen": "QUIROFANO_01"
  }'
```

2. **Consultar Stock Disponible**

```
curl -X GET "http://servidor:3000/api/epc-readings/stock" \
  -H "x-api-key: API_KEY"
```

- Filtrado por almacén:

```
curl -X GET "http://servidor:3000/api/epc-readings/stock?devid=QUIROFANO_01" \
  -H "x-api-key: API_KEY"
```

3. **Buscar Instrumental Específico**

- Por EPC:

```
curl -X GET "http://servidor:3000/api/epc-master/epc/E2000016600001234567890 1" \
  -H "x-api-key: API_KEY"
```

- Por descripción parcial:

```
curl -X GET "http://servidor:3000/api/epc-master/description/bisturi" \
  -H "x-api-key: API_KEY"
```

4. **Actualizar Información del Instrumental**

```
curl -X PUT "http://servidor:3000/api/epc-master/1" \
  -H "x-api-key: API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "short_description": "Bisturí quirúrgico modificado",
    "almacen": "QUIROFANO_02"
  }'
```

5. **Eliminar Registro**

```
curl -X DELETE "http://servidor:3000/api/epc-master/1" \
  -H "x-api-key: API_KEY"
```

3.2 Seguimiento de Consumo

Escenario

Rastrear qué instrumental se ha utilizado en cada cirugía para control de stock y trazabilidad.

Flujo de Trabajo

1. **Consultar Consumo por Paciente o Quirófano**

```
curl -X GET "http://servidor:3000/api/bin/stock-consumption?
patient=PAC001&operating_room=QUI01" \
  -H "x-api-key: API_KEY"
```

2. **Consultar Consumo por Rango de Fechas**

```
curl -X GET "http://servidor:3000/api/bin/stock-consumption?startDate=2024-01-01&endDate=2024-
01-31" \
  -H "x-api-key: API_KEY"
```

3. **Generar Reporte Agregado de Consumo**

```
// JavaScript ejemplo
async function generarReporteConsumo(fechaInicio, fechaFin) {
  const response = await fetch(`/api/bin/stock-consumption?startDate=${fechaInicio}&endDate=$
{fechaFin}`, {
    headers: { 'x-api-key': API_KEY }
  });
  const consumo = await response.json();
  const reporte = consumo.reduce((acc, item) => {
    const key = item.EPC;
    if (!acc[key]) acc[key] = { epc: key, cantidad: 0, pacientes: new Set() };
    acc[key].cantidad++;
    acc[key].pacientes.add(item.patient);
    return acc;
  }, {});
  return Object.values(reporte);
}
```

3.3 Control de Acceso

Escenario

Monitorear el acceso de personal y restringir entrada a áreas críticas.

Flujo de Trabajo

1. **Registrar Usuario en Sistema**

```
curl -X POST "http://servidor:3000/api/user-room" \
  -H "x-api-key: API_KEY" \
  -H "Content-Type: application/json" \
```

```
  -d '{
    "epc": "E2000016600009876543210 98",
    "door": "QUIROFANO_01",
    "state": 1
  }'
```

2. **Consultar Accesos por Puerta**

```
curl -X GET "http://servidor:3000/api/user-room/door/QUIROFANO_01" \
  -H "x-api-key: API_KEY"
```

3. **Consultar Historial de Accesos**

```
curl -X GET "http://servidor:3000/api/user-readings/user-door?devid=DOOR_001&startDate=2024-
01-01" \
  -H "x-api-key: API_KEY"
```

3.4 Integración con Sistema Hospitalario

Escenario

Conectar la API RFID con ERP/HIS del hospital para sincronización de inventario y trazabilidad.

Ejemplo de Integración en Python

```
import requests

class RFIDIntegration:
    def __init__(self, base_url, api_key):
        self.base_url = base_url
        self.headers = {'x-api-key': api_key, 'Content-Type': 'application/json'}

    def sincronizar_inventario(self):
        stock = requests.get(f"{self.base_url}/epc-readings/stock",
headers=self.headers).json()
        inventario_completo = []
        for item in stock:
            detalle = requests.get(f"{self.base_url}/epc-master/epc/{item['EPC']}",
headers=self.headers).json()
            if detalle:
                inventario_completo.append({
                    'epc': item['EPC'],
                    'descripcion': detalle[0]['short_description'],
                    'almacen': detalle[0]['Almacen'],
                    'ultima_lectura': item['Local_time']
                })
        return inventario_completo
```

3.5 Mantenimiento y Limpieza

Flujo de Trabajo

1. **Identificar Instrumental para Mantenimiento**
2. **Marcar Instrumental como Fuera de Servicio**

3. **Consultar Historial de Uso para Planificación**

3.6 Auditoría y Compliance

Escenario

Generar reportes para auditoría y cumplimiento normativo, integrando inventario, consumo y accesos.

Ejemplo en JavaScript

```
class AuditoriaRFID {
  constructor(apiUrl, apiKey) { this.apiUrl = apiUrl; this.apiKey = apiKey; }
  async generarReporteCompleto(inicio, fin) {
    const inventario = await fetch(`${this.apiUrl}/epc-master`, { headers: {'x-api-key':
this.apiKey}}).then(r => r.json());
    const consumo = await fetch(`${this.apiUrl}/bin/stock-consumption?startDate=$
{inicio}&endDate=${fin}`, { headers: {'x-api-key': this.apiKey}}).then(r => r.json());
    const accesos = await fetch(`${this.apiUrl}/user-readings/user-door?startDate=$
{inicio}&endDate=${fin}`, { headers: {'x-api-key': this.apiKey}}).then(r => r.json());
    return { inventario, consumo, accesos };
  }
}
```

---

4. Preguntas Frecuentes (FAQ) – API RFID

Configuración y Acceso

- **¿Cómo obtengo una API Key?** Solicita al administrador del sistema.
- **¿Puedo usar la misma API Key en múltiples aplicaciones?** Sí, pero se recomienda usar claves distintas.
- **¿Expira la API Key?** No, salvo revocación manual.

Conectividad y Rendimiento

- **Límites de llamadas:** Recomendado ≤100 llamadas/min.
- **Soporte HTTPS:** Sí, obligatorio en producción.
- **Timeout:** 30 segundos por defecto.

Datos y Formatos

- **Formato de fecha:** `YYYY-MM-DD` o `YYYY-MM-DDTHH:mm:ss.sssZ`.
- **Formato EPC:** String hexadecimal. Ej.: `E20000166000012345678901`.
- **Campos sensibles a mayúsculas/minúsculas.**

Funcionalidades

- **Stock vs Stock Temporal:**

    - Stock: disponible en almacén
    - Stock Temporal: en tránsito o en proceso
- **Recuperar registros eliminados:** No, DELETE es permanente
- **Búsqueda parcial por descripción:** `/epc-master/description/{texto}`

Errores Comunes

- **401 Unauthorized:** API Key no proporcionada
- **403 Forbidden:** API Key inválida
- **404 Not Found:** Endpoint inexistente
- **500 Internal Server Error:** Error de servidor

Integración

- **Lenguajes soportados:** Cualquier cliente HTTP REST (JavaScript, Python, PHP, etc.)
- **Webhooks:** No